Exp-03: Multi-Level Cache Design

Tags

Multi-Level Cache Design

Objective

So far, we have studied direct-mapped and 2-way set-associative caches. We now move from designing **one cache** to understanding how **multiple cache levels work together**.

You will investigate:

> **How does the memory-access behavior of a workload influence the design and performance of a multi-level cache hierarchy?**

The focus is on connecting:

Access Pattern → Locality → Cache Mapping → Hit/Miss → Cache Level → Performance

---

1. System to Simulate

Use the following simplified processor:

```
   CPU
    |
    v
   L1
    |
L1 Miss
    |
    v
   L2
    |
L2 Miss
    |
    v
   L3
    |
L3 Miss
    |
    v
  DRAM
```

Main Memory

Parameter	Value
Capacity	64 MB
Address size	26 bits
Addressable unit	1 byte
DRAM latency	100 cycles

Cache Hierarchy

Level	Capacity	Line Size	Baseline Mapping	Access Latency
L1	32 KB	64 B	Direct mapped	1 cycle
L2	256 KB	64 B	2-way	5 cycles
L3	2 MB	64 B	4-way	20 cycles

The cache-line size and capacities remain fixed throughout the experiment.

You may change associativity between:

1-way, 2-way, 4-way and 8-way, wherever valid.

---

2. Cache Access Must Be Hierarchical

Every CPU memory request must follow this order:

Do not randomly choose a cache level for an access.

A request reaches L2 **only after an L1 miss**.

A request reaches L3 **only after both L1 and L2 miss**.

A request reaches DRAM **only after all three cache levels miss**.

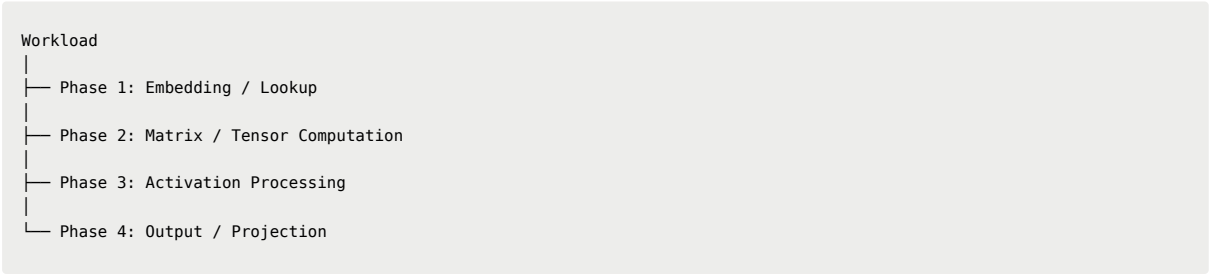
When data is found at a lower level, model the appropriate movement back toward the processor.

---

3. Workload

Instead of running two unrelated programs, simulate **one simplified workload containing multiple phases**.

This is intentional: a real workload does not have one single memory-access pattern. Different stages can behave very differently.



The phases should be executed **in this order as one workload**.

Do not clear the caches between phases unless your implementation specifically requires it. This allows you to observe how one phase can influence the cache state seen by the next phase.

---

4. Phase 1 — Embedding / Lookup

Use an embedding table of:

- 1,048,576 entries
- 16 values per entry
- 4 bytes per value

Total size:

64 MB

Use a non-sequential index stream such as:

```
17, 2048, 91, 17, 8192, 2048, 503, 91, ...
```

The important characteristic is **irregular access**.

Expected properties:

- weak spatial locality between different lookups
- some temporal reuse
- potentially high cache pressure

Do not assume the outcome. Measure it.

---

5. Phase 2 — Matrix/Tensor Computation

Use a matrix multiplication-style computation:

```
C = A × B
```

where:

```
A = 256 × 256
B = 256 × 256
C = 256 × 256
```

Each element is 4 bytes.

Therefore:

- A = 256 KB
- B = 256 KB
- C = 256 KB
- Total = **768 KB**

Use a loop structure equivalent to:

```
for i
    for j
        for k
            C[i][j] += A[i][k] * B[k][j]
```

This produces a mixture of:

- sequential access
- strided access
- repeated access

- spatial locality
- temporal locality

Pay particular attention to how the different access patterns interact with the cache.

---

6. Phase 3 — Activation Processing

Process a large tensor sequentially:

```
for i = 0 to N
    output[i] = activation(input[i])
```

The important characteristic is **streaming/sequential access**.

This gives strong spatial locality, but relatively little temporal reuse because an element may be consumed and never accessed again.

This phase is useful for investigating whether **good spatial locality necessarily means that keeping data in the cache provides a large performance benefit**.

---

7. Phase 4 — Output / Projection

Use another matrix/tensor-style operation representing the final projection/output stage.

It should contain a mixture of:

- sequential accesses
- reused data
- strided accesses

The purpose is to create another change in memory behavior before the inference completes.

---

8. Your Experimental Task

Step 1 — Run the Baseline

Start with:

```
L1 = 1-way
L2 = 2-way
L3 = 4-way
```

Run the **complete four-phase inference workload**.

Record the cache behavior.

Step 2 — Analyze the Access Patterns

For each phase, identify:

- sequential accesses
- strided accesses
- irregular accesses
- repeated accesses

- spatial locality
- temporal locality

Explain how you expect these patterns to affect the hierarchy.

Step 3 — Design Alternative Hierarchies

Create and test **at least three complete cache configurations**.

For example:

```
Configuration A
L1 = 1-way
L2 = 2-way
L3 = 4-way

Configuration B
L1 = 2-way
L2 = 4-way
L3 = 8-way

Configuration C
L1 = 4-way
L2 = 4-way
L3 = 8-way
```

These are examples, not predetermined answers.

You should have a reason for choosing the configurations you test.

Keep everything else unchanged.

---

9. Measure

For every configuration, collect at least:

- L1 hits and misses
- L2 hits and misses
- L3 hits and misses
- DRAM accesses
- L1/L2/L3 miss rates
- AMAT or total memory-access cycles
- overall workload execution time, if available

Also record results **phase-wise**.

For example:

Phase	L1 Miss %	L2 Miss %	L3 Miss %	DRAM Accesses	Cycles
Embedding					
Matrix					
Activation					
Projection					

And compare the complete configurations:

Configuration	L1 Miss %	L2 Miss %	L3 Miss %	DRAM Accesses	AMAT	Runtime
A						

Configuration	L1 Miss %	L2 Miss %	L3 Miss %	DRAM Accesses	AMAT	Runtime
B						
C						

10. Analyze What Happened

Your analysis should explain the relationship:

```
Workload Phase
      ↓
Memory Access Pattern
      ↓
Locality
      ↓
Cache Mapping
      ↓
Hit / Miss Behavior
      ↓
Level at which data is found
      ↓
Memory Access Cost
      ↓
Overall Performance
```

Look specifically for:

- conflict misses
- changes caused by associativity
- phases that benefit from the cache
- phases that generate substantial cache traffic
- whether increasing associativity actually improves performance
- whether an improvement at one level affects traffic at another level
- whether a cache configuration that helps one phase is necessarily best for the complete workload
- whether one phase changes the cache state used by the following phase

11. Final Architectural Decision

Choose the cache hierarchy you would recommend:

```
L1 = ?-way
L2 = ?-way
L3 = ?-way
```

Justify your decision using your measurements.

Do **not** select a configuration simply because it has the highest hit rate.

Explain why the observed performance occurred and whether the architectural change was actually worthwhile.

Also identify **one result that surprised you** and explain what it taught you about cache behavior.

12. What to Submit

Keep the report concise and include:

1. **Cache hierarchy and system parameters**
2. **Description of the four workload phases**
3. **Access-pattern and locality analysis**
4. **Baseline results**
5. **At least three cache hierarchy configurations**
6. **Experimental results**
7. **Important observations**
8. **Architectural explanation of those observations**
9. **Recommended L1/L2/L3 configuration**
10. **One surprising result and what you learned from it**

The central idea

The purpose of this assignment is **not** to prove that a particular associativity is always better.

The important question is:

> **How does the behavior of the workload determine whether an architectural change is useful?**

A cache hierarchy is a collection of trade-offs. Different phases of the same workload can place very different demands on it.

Your job is to use the experimental evidence to explain **why the hierarchy behaved the way it did** and make an architectural decision based on that evidence.

The deeper questions behind this experiment—such as why L1 and L3 might be designed differently, why we do not make every cache fully associative, and what trade-offs architects are making—will be explored during the class discussion.